

UF3

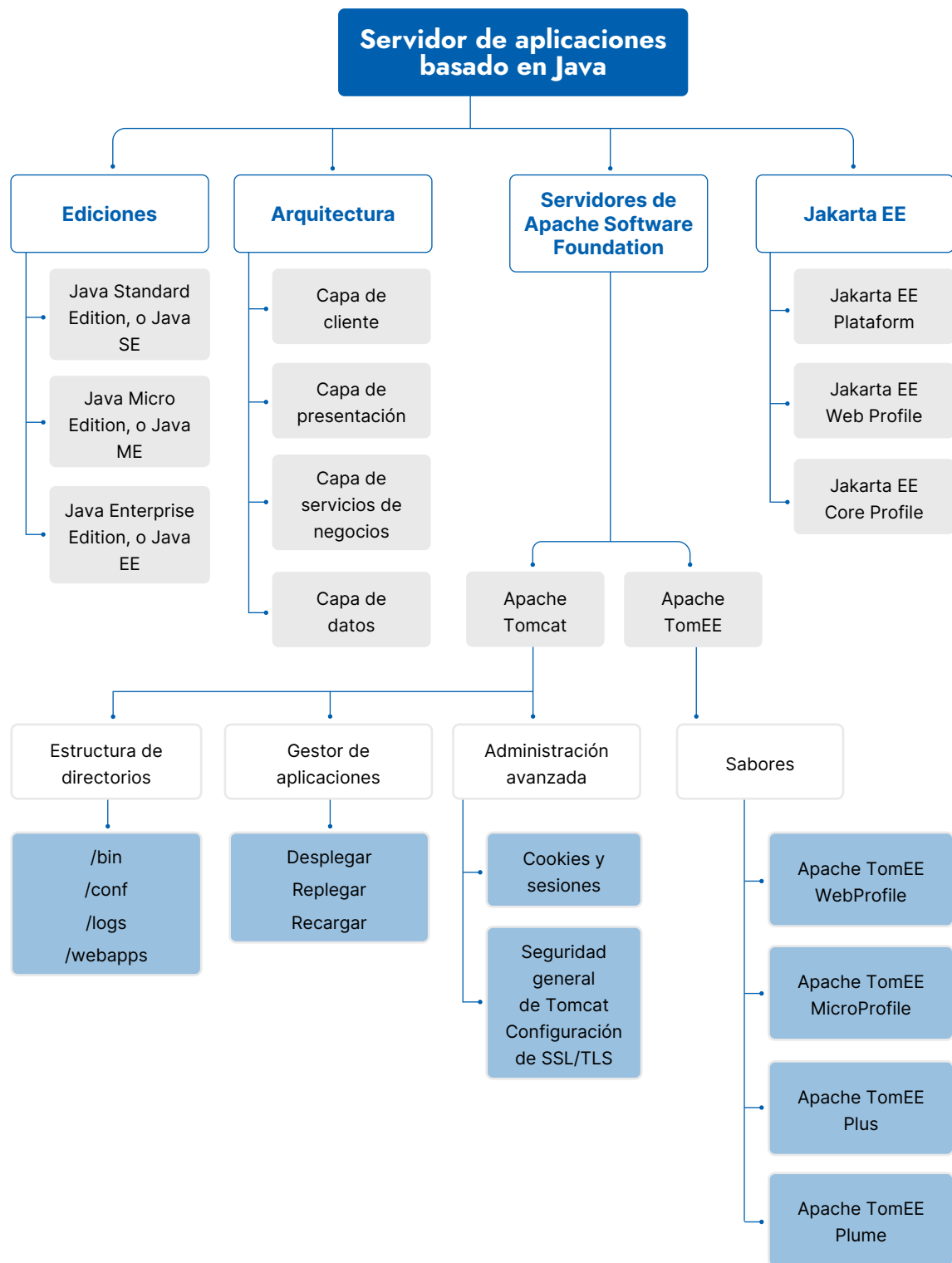
Servidores de aplicaciones

Despliegue de aplicaciones web

ÍNDICE

Mapa conceptual	03
1. Introducción a los servidores de aplicaciones	04
1.1. Lenguaje de programación Java	05
1.2. Desarrollo de aplicaciones Java	06
A. Plataforma Java de Oracle.....	06
B. Especificaciones Jakarta EE	07
C. Servidores de Apache Software Foundation	08
2. Apache Tomcat.....	12
2.1. Estructura de directorios	12
2.2. Variables	13
2.3. Aplicaciones	14
2.4. Gestor de aplicaciones de Tomcat	15
2.5. JSP y servlets Java.....	17
2. Apache Tomcat avanzado	18
3.1. Cookies	18
3.2. Manejo de sesiones.....	20
3.3. Seguridad	22
A. Seguridad general de Tomcat	22
B. Configuración de SSL/TLS	23

Mapa conceptual



01 Introducción a los servidores de aplicaciones

Como se vio en la unidad 2, los **servidores web** proporcionan páginas web estáticas y, gracias a los lenguajes de programación o de *scripting* como PHP, JSP, ASP o Perl, también páginas dinámicas. Si a esto le sumamos la capacidad de usar sistemas de archivos y de comunicarse con los sistemas gestores de bases de datos, los servidores web son capaces de albergar aplicaciones web complejas con las mismas funcionalidades que otro tipo de aplicaciones.

Los **servidores de aplicaciones** comparten su mismo propósito, es decir, la ejecución remota de aplicaciones, solo que, en este caso, también pueden ejecutar programas escritos en algunos lenguajes de programación tradicionales, como Java, Python, C o C++. Por lo tanto, los servidores de aplicaciones permiten la ejecución de aplicaciones web, al igual que los servidores web, pero, además, ofrecen una interfaz entre el usuario y la aplicación alojada en el servidor, también conocida como **lógica de negocio**, que permite atender las peticiones de los clientes y enviarles la correspondiente respuesta. La aplicación podrá acceder a los datos si es necesario.

Las **ventajas** que aportan los servidores de aplicaciones frente a un modelo descentralizado son las siguientes:

- » Ejecución de programas desde equipos que normalmente no podrían ejecutarlos por incompatibilidades con el sistema operativo local o por la falta de recursos de hardware.
- » Tener clientes con menores especificaciones de hardware, por lo que baja la inversión en ellos.
- » Disminución de la administración de los clientes.
- » La centralización de los datos permite mejorar su tratamiento y protección.
- » Facilidad a la hora de administrar las actualizaciones y el despliegue de los programas.

Por contra, este modelo también presenta algunas **desventajas**:

- » Se necesitan servidores más potentes.
- » Al transmitirse la información por la red, la superficie de ataque del sistema aumenta, por lo que hay que destinar recursos a la protección de la información en tránsito.
- » Si cae el servicio o todo el servidor, ninguno de los clientes podrá trabajar, por lo que se deben implementar soluciones de recuperación ante caídas (*failover*).
- » Si no se dimensiona bien el servidor, puede que este se sature ante las peticiones de los clientes. Es aconsejable el uso de técnicas que eviten esta situación, como, por ejemplo, el balanceo de carga.

Un escenario típico en el que utilizar un servidor de aplicaciones sería una empresa que tiene un programa de gestión centralizado en un servidor de aplicaciones dentro de la red corporativa. Los usuarios conectados a la red acceden a esta aplicación a través de un cliente, que, en la mayoría de los casos, se trata de un navegador web. El uso de herramientas de escritorio remoto, o VPN, permitiría el acceso a trabajadores que se encuentren en otras sedes de la empresa, en las instalaciones de los clientes o en sus propios domicilios si están teletrabajando.

1.1. Lenguaje de programación Java

Hoy en día es muy común encontrar servidores de aplicaciones que ejecutan aplicaciones escritas en Java, por lo que es interesante conocer algunos de sus aspectos más importantes. Cuando se habla de Java, se puede hacer referencia tanto al lenguaje de programación como a la plataforma necesaria para trabajar con los programas. Centrándonos en el lenguaje, diremos que es multiplataforma, está orientado a objetos, e integra muchas funciones y bibliotecas.

A diferencia de otros lenguajes de programación de alto nivel compilados, Java, en vez de generar código máquina entendible directamente por la máquina para la que se ha compilado, genera un código intermedio llamado Java *bytecode*, que necesita una máquina virtual de Java (JVM) instalada para ejecutarse. En realidad, Java compila para generar el *bytecode* que, luego, la JVM interpreta cada vez que se ejecuta la aplicación.

Este procedimiento, que, a primera vista, puede parecer una desventaja, en realidad permite compilar una vez el programa escrito en Java y ejecutarlo en cualquier sistema con la máquina virtual de Java instalada, mientras que, para un lenguaje compilado, como C o C++, debe obtenerse el código máquina para cada tipo de dispositivo o sistema operativo.

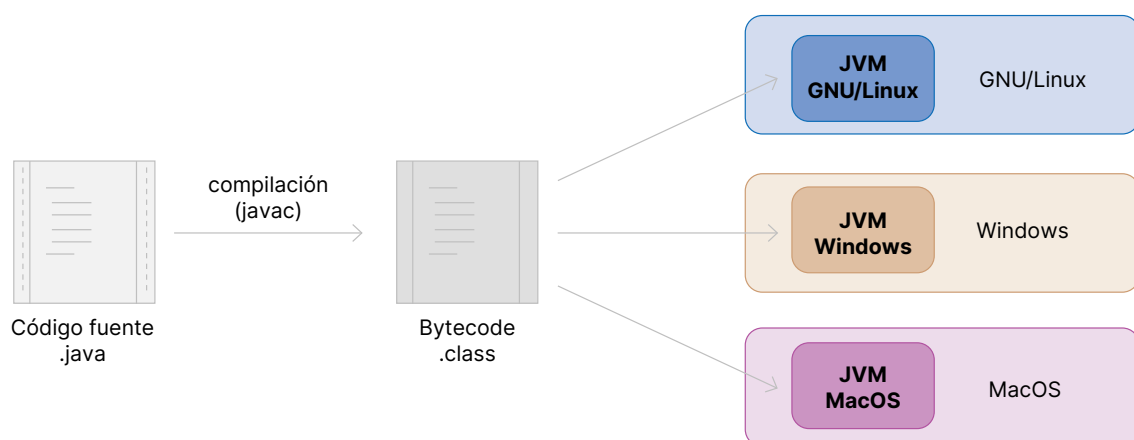


Figura 1. Proceso de lenguaje de programación Java.

1.2. Desarrollo de aplicaciones Java

Existen varias plataformas de desarrollo para Java. Dependiendo del alcance del software que se quiera desarrollar, será conveniente usar unas u otras.

A. Plataforma Java de Oracle

Esta plataforma, gestionada por Oracle, pero creada por Sun Microsystems, aglutina un gran número de tecnologías que ofrecen herramientas muy potentes para el desarrollo de aplicaciones programadas en Java. Estas son sus diferentes ediciones:

- » **Java Standard Edition, o Java SE**, es la plataforma básica que ofrece Oracle para implementar aplicaciones, ya sean de escritorio o de servidor. Existe una versión gratuita y de código abierto llamada OpenJDK que contiene los principales componentes, como la JVM, las librerías o el compilador de Java.
- » **Java Micro Edition, o Java ME**, es una versión reducida de Java SE a la que se le han incorporado bibliotecas específicas para dispositivos móviles y sistemas integrados.
- » **Java Enterprise Edition, o Java EE**, además de las funciones de Java SE, ofrece herramientas para poder desarrollar aplicaciones más grandes y complejas con aspectos avanzados como tolerancia a fallos y computación distribuida. En 2017, las tecnologías Java EE se trasladaron a la Eclipse Foundation, donde continúa su desarrollo bajo la marca Jakarta EE.

Uno de los aspectos fundamentales de la **arquitectura** de las plataformas Java es la independencia de sus componentes, lo que se consigue gracias a su diseño dividido en las siguientes **capas lógicas**:

- » **Capa de cliente**: elementos que se ejecutan en la máquina cliente. Normalmente a través de un navegador web, aplicaciones de escritorio o para móvil.
- » **Capa de presentación**: está en el servidor Java EE o servidor de aplicaciones y su función es procesar las peticiones de la capa cliente para enviarla a la capa de servicios de negocio y preparar las respuestas de manera que transforma la información a un formato legible por el cliente, normalmente HTML. Por regla general, se compone de servlets y JSP (JavaServer Pages).
- » **Capa de servicios de negocios**: también se ejecuta en el servidor de aplicaciones y su tarea principal es realizar las funciones principales de las aplicaciones, proporcionar los datos o guardarlos en la capa de datos.
- » **Capa de datos**: servicios que proporcionan acceso a la información almacenada habitualmente en una base de datos o servicios de directorios como LDAP (siglas inglesas de *Lightweight Directory Access Protocol*).

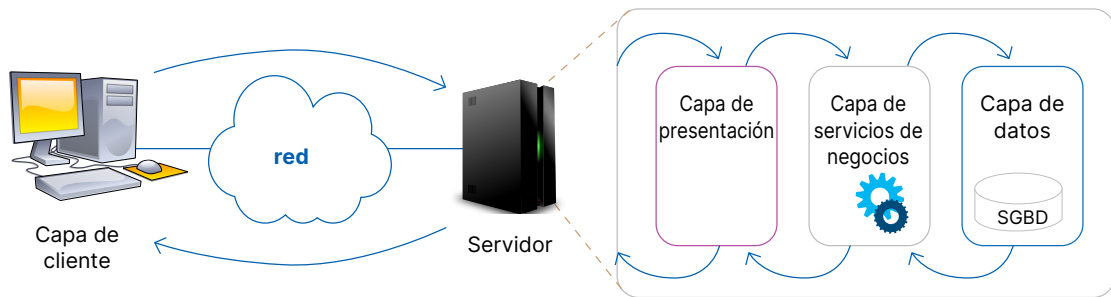


Figura 2. Funcionamiento de aplicaciones Java.

B. Especificaciones Jakarta EE

Jakarta EE facilita un conjunto de especificaciones gestionadas por **Eclipse Foundation** que constituyen un marco de código abierto para que empresas y organizaciones puedan seguirlas a la hora de **crear sus plataformas de desarrollo**. En ese caso, si una empresa quiere que su producto se certifique como compatible con Jakarta EE, debe seguir un proceso que empieza con una solicitud de compatibilidad. En la sección *Compatible products* de su página oficial se puede obtener la lista de productos compatibles e, incluso, descargarlos. De esta manera, los desarrolladores de software pueden obtener plataformas de desarrollo que cumplan con los criterios de calidad, seguridad, estabilidad y flexibilidad marcados por Jakarta EE. Algunas de las plataformas de desarrollo certificadas por Jakarta EE son Apache TomEE, Eclipse GlassFish, IBM WebSphere Liberty o JBoss Enterprise Application Platform.

Jakarta EE tiene tres especificaciones marco que recogen especificaciones individuales. Estas especificaciones generales son las siguientes:

Jakarta EE Platform	Incluye la mayoría de las especificaciones individuales para definir cómo debería ser una plataforma para alojar aplicaciones.
Jakarta EE Web Profile	Contiene las especificaciones individuales orientadas al desarrollo de plataformas web.
Jakarta EE Core Profile	Recoge aquellas especificaciones necesarias para desarrollar arquitecturas de microservicios.

Las especificaciones individuales definen aspectos particulares. A continuación, se presentan algunas, pero se puede encontrar la lista completa en la sección *Specifications* de la página oficial de Jakarta EE.

<i>Jakarta Servlet</i>		API para la gestión de las peticiones y respuestas HTTP.
<i>Jakarta Server Pages</i>		Define un motor para aplicaciones web.
<i>Jakarta Expression Language</i>		Lenguaje de expresiones para aplicaciones Java.
<i>Jakarta WebSocket</i>		Uso de <i>websockets</i> en las aplicaciones de cliente y servidor.
<i>Jakarta Annotations</i>		Representación semántica para la programación declarativa.
<i>Jakarta Authentication</i>		Definición de mecanismos de autenticación.
<i>Jakarta Enterprise Beans</i>		Arquitectura y componentes para aplicaciones.
<i>Jakarta Enterprise Web Services</i>		Define servicios web.
<i>Jakarta Faces</i>		Diseño de interfaces web de usuario.

C. Servidores de Apache Software Foundation

Como ya se ha apuntado, Jakarta EE es un conjunto de especificaciones en las que las organizaciones y empresas pueden basarse a la hora de crear sus plataformas de desarrollo. Apache Software Foundation tiene dos implementaciones en Jakarta EE:

» **Apache Tomcat:** implementación de código abierto de algunas especificaciones Jakarta EE, como Jakarta Servlet, Jakarta Server Pages, Jakarta Expression Language, Jakarta WebSocket, Jakarta Annotations y Jakarta Authentication. Hay que tener en cuenta que Tomcat no cumple con todas las especificaciones indicadas por Jakarta EE para ser considerado un servidor de aplicaciones, por lo que se trata de un **servidor de servlets**. Aun así, la mayoría de las aplicaciones sencillas pueden funcionar perfectamente sobre él, ya que es capaz de manejar servlets y JSP. Tomcat 10 y las versiones posteriores implementan especificaciones desarrolladas como parte de Jakarta EE, mientras que Tomcat 9 y las versiones anteriores se han desarrollado como parte de Java EE.

» **Apache TomEE:** es un servidor de aplicaciones completo y certificado por Jakarta EE 9.1. Amplía las especificaciones recogidas por Tomcat. Apache TomEE tiene distintos sabores, de manera que cada uno cuenta con las funciones del sabor anterior y añade alguna más. Estos sabores son los siguientes:

- **Apache TomEE WebProfile**, que, además de contar con las funciones de Tomcat, añade otras como la persistencia de datos y servicios web.
- **Apache TomEE MicroProfile**, que añade una arquitectura de microservicios.
- **Apache TomEE Plus**, que incluye, entre otras características, la autorización, mensajería o gestión de servicios en segundo plano.
- **Apache TomEE Plume**, por su parte, no aporta mejoras en las características, sino que incluye otras implementaciones de interfaces web de usuario y acceso a datos.

En la página oficial de Apache TomEE hay un enlace que lleva a una comparativa más exhaustiva de estos sabores y de las especificaciones Jakarta EE que cumplen.



VÍDEOLAB

Para consultar el siguiente vídeo sobre autenticación de aplicaciones, escanea el código QR o [pulsa aquí](#).

1

CASO PRÁCTICO



INSTALACIÓN DE TOMCAT

Como se ha descrito en la teoría, Tomcat no es un servidor de aplicaciones, sino un contenedor web con soporte de servlets y JSP. Para realizar su instalación debes dar los siguientes pasos:

1. Instalar JDK.
2. Crear un usuario y un grupo llamados *tomcat*, cuyo directorio de inicio del usuario sea */opt/tomcat*.
3. Instalar Tomcat.
4. Habilitar el acceso al gestor de aplicaciones.
5. Habilitar el rol de gestión para acceder al gestor de aplicaciones.
6. Crear el servicio de Tomcat.
7. Iniciar y activar el servicio de Tomcat.
8. Comprobar que Tomcat funciona a través del navegador en el puerto 8080.

SOLUCIÓN

Dada la complejidad de esta actividad, el docente puede optar por dar al alumnado parte de la solución o las líneas de código que crea oportunas. A continuación se describen los pasos que se deben seguir para instalar la versión de Tomcat 10.0.22, última versión disponible a la hora de escribir este manual:

1. Para compilar y ejecutar programas en Java, es necesario tener instalado el kit de desarrollo de Java (JDK). En este caso, se ha optado por instalar la versión de código abierto OpenJDK. Para instalarlo, tienes que abrir una sesión con un **usuario administrador** y ejecutar el siguiente comando:

```
$ sudo apt install default-jdk -y
```

2. Ahora vas a crear el grupo y usuario *tomcat* que administrará esta aplicación. Además, tendrá acceso a un *shell* y, aunque no es recomendable en un servidor en producción, para estas pruebas tendrá acceso remoto. Para ello, tienes que ejecutar la siguiente instrucción:

```
$ sudo adduser --home /opt/tomcat --gecos "" tomcat
```

3. Para instalar Tomcat, debes abrir una sesión con el **usuario tomcat**, comprobar que estás en su directorio personal (*/opt/tomcat*), descargar la última versión desde su repositorio oficial y descomprimirlo. No olvides eliminar el archivo comprimido cuando ya no lo necesites. A continuación, tienes los comandos necesarios:

```
$ wget https://dlcdn.apache.org/tomcat/tomcat-10/v10.0.22/bin/apache-tomcat-10.0.22.tar.gz
```

```
$ tar xzvf apache-tomcat-10*tar.gz
```

```
$ rm apache-tomcat-10.0.22.tar.gz
```

4. Tras la instalación de Tomcat, el acceso al gestor de aplicaciones está restringido, por lo que debes conectarte como **administrador** y eliminar las restricciones de acceso por defecto porque solo permiten la conexión local. Hay que tener en cuenta que en un sistema en explotación se debería limitar el acceso a las direcciones IP imprescindibles. Para eliminar las restricciones hay que editar el archivo *webapps/manager/META-INF/context.xml* con el comando:

\$ sudo nano /opt/tomcat/apache-tomcat-10.0.22/webapps/manager/META-INF/context.xml
y comentar o eliminar las líneas siguientes (recuerda que los comentarios en html se abren con `<!--` y se cierran con `-->`):

```
<Valve className="org.apache.catalina.valves.RemoteAddrValve"
allow="»127\.\d+\.\d+\.\d+|::1|0:0:0:0:0:0:0:1" />
```

5. Para poder conectarte al gestor de aplicaciones de Tomcat desde su página web, debes crear una cuenta de usuario de Tomcat con este rol en el archivo *tomcat-users.xml*, por tanto, tienes que acceder al archivo con el siguiente comando:

```
$ sudo nano /opt/tomcat/apache-tomcat-10.0.22/conf/tomcat-users.xml
```

Y, una vez abierto el archivo, añade estas líneas justo antes de la etiqueta `</tomcat-users>`

```
<user username="admin" password="admin" roles="manager-gui"/>
```

Como puedes comprobar, el usuario y la contraseña son *admin*. Puedes cambiarlos por los que desees.

6. Una vez instalado Tomcat, hay que indicarle al sistema que lo ejecute como un servicio. Para ello, crea el archivo */etc/systemd/system/tomcat.service* con tu editor de texto preferido desde la consola del **usuario administrador**.

```
$ sudo nano /etc/systemd/system/tomcat.service
```

y escribe el siguiente contenido:

```
[Unit]
```

```
Description=Tomcat servlet container
```

```
After=network.target
```

```
[Service]
```

```
Type=forking
```

```
User=tomcat
```

```
Group=tomcat
```

```
Environment="JAVA_HOME=/usr/lib/jvm/default-java"
```

```
Environment="JAVA_OPTS=-Djava.security.egd=file:///dev/urandom"
```

```
Environment="CATALINA_BASE=/opt/tomcat/apache-tomcat-10.0.22"
```

```
Environment="CATALINA_HOME=/opt/tomcat/apache-tomcat-10.0.22"
```

```
Environment="CATALINA_PID=/opt/tomcat/apache-tomcat-10.0.22/temp/tomcat.pid"
```

```
Environment="CATALINA_OPTS=-Xms512M -Xmx1024M -server -XX:+UseParallelGC"
```

```
ExecStart=/opt/tomcat/apache-tomcat-10.0.22/bin/startup.sh
```

```
ExecStop=/opt/tomcat/apache-tomcat-10.0.22/bin/shutdown.sh
```

```
[Install]
```

```
WantedBy=multi-user.target
```

El estudio de *systemd* queda fuera del objetivo de esta publicación, por lo que no se explicará el contenido de este archivo. No obstante, puedes buscar más información en su página oficial o en las páginas del *man*.

7. Para iniciar y activar el servicio Tomcat de manera que se active cuando se levante el sistema, ejecuta los siguientes comandos:

```
sudo systemctl start tomcat.service
```

```
sudo systemctl enable tomcat.service
```

8. Por último, solamente queda comprobar que Tomcat funciona a través del navegador, indicando la IP del servidor y el puerto 8080.

```
http://IP_servidor:8080
```

02 Apache Tomcat

Apache Tomcat es un contenedor web con soporte de servlets y JSP que incluye el compilador Jasper para convertir archivos JSP en servlets. El nombre del contenedor de servlets de Tomcat a partir de la versión 4.x es Catalina y el puerto por defecto en el que funciona Tomcat es el 8080/TCP.

2.1. Estructura de directorios

Con la instalación de Tomcat se crean varios directorios y archivos. A continuación, se describen los directorios de la versión 10 de Tomcat:

bin	Este directorio contiene scripts, entre ellos, los de inicio y parada. También contiene los archivos *.sh (en sistemas Unix) o *.bat (en sistemas Windows).
conf	Archivos de configuración. El archivo más importante es server.xml , que es el archivo de configuración principal del contenedor.
lib	Directorio que contiene librerías.
logs	Directorio que contiene los archivos de registro.
temp	Utilizado por la JVM para archivos temporales.
webapps	Directorio donde están las aplicaciones.
work	Contiene directorios de trabajo temporales para las aplicaciones web implementadas.

Tabla 1. Estructura de directorios.

2.2. Variables

En un mismo servidor de Tomcat pueden ejecutarse varias instancias y las variables **CATALINA_HOME** y **CATALINA_BASE** son las que permiten trabajar de esta forma. A continuación, se explica cómo utilizarlas correctamente:

- » **CATALINA_HOME**: apunta al directorio raíz de la instalación de Tomcat.
- » **CATALINA_BASE**: apunta al directorio raíz de configuración de una instancia específica de Tomcat en tiempo de ejecución. Se utiliza para tener varias instancias de Tomcat en una máquina.

Por defecto, **CATALINA_HOME** y **CATALINA_BASE** apuntan al mismo directorio. Si se necesita ejecutar varias instancias de Tomcat en una máquina, se ha de configurar **CATALINA_BASE** manualmente. En este caso, **CATALINA_HOME** contendrá los elementos comunes a todas las instancias, como archivos .jar o archivos binarios, mientras que **CATALINA_BASE** contendrá archivos de configuración, archivos de registro, aplicaciones implementadas y otros requisitos de tiempo de ejecución correspondientes a la instancia. De esta forma se conseguirá:

- » Actualizar fácilmente todas las instancias a una versión más nueva de Tomcat, ya que, al depender todas de una sola ubicación **CATALINA_HOME**, comparten los archivos .jar y archivos binarios.
- » Evitar la duplicación de los mismos archivos .jar estáticos.
- » La posibilidad de compartir ciertas configuraciones, como funciones del *shell* o scripts *bat* (dependiendo del sistema operativo).

Para cada instancia, será necesario replicar el árbol de directorios apuntado por **CATALINA_HOME** en la ubicación apuntada por **CATALINA_BASE**. Una opción es que Tomcat lo cree automáticamente, pero puede fallar si no tiene los permisos adecuados y, por tanto, la instancia puede que no funcione correctamente, por lo que es más seguro crearla manualmente. Además de crear los directorios, puede que sea necesario copiar algunos archivos. A continuación se describen algunas recomendaciones:

DIRECTORIO	RECOMENDACIONES
bin	Este directorio contiene ejecutables y scripts. Si se desea una implementación de registro personalizada, además de crear el directorio, se deberán copiar los archivos siguientes: setenv.sh , setenv.bat y tomcat-juli.jar .
conf	Archivos de configuración del propio Tomcat y de parámetros por defecto para las aplicaciones.
lib	Si la aplicación depende de librerías externas, se deberán agregar más recursos en <i>classpath</i> .

Tabla 2. Recomendaciones para variables.

2.3. Aplicaciones

Se dice que una aplicación se encuentra **desempaquetada** cuando se puede acceder a sus archivos, organizados en directorios, de forma directa. Una aplicación también puede estar empaquetada en un archivo .war, un formato más cómodo para su distribución.

Como se ha visto en el punto anterior, todas las aplicaciones deben estar contenidas en el directorio **webapps**. A su vez, cada una de ellas tendrá su propio directorio, cuyo nombre suele coincidir con el de la aplicación. Su estructura de directorios y archivos es la siguiente:

- » **Directorio raíz de la aplicación:** páginas HTML, páginas JSP, archivos JavaScript, archivos CSS, imágenes, etc.
- » **/WEB-INF/web.xml:** este archivo describe los servlets y los componentes de la aplicación. También puede contener parámetros de inicialización y restricciones de seguridad.
- » **/WEB-INF/classes/:** en este directorio se ubican los archivos de Java .class.
- » **/WEB-INF/lib/:** en este directorio se guardan los archivos .jar, que contienen librerías o controladores JDBC (del inglés Java Database Connectivity), en caso de necesitarlos.

En la siguiente imagen se puede ver la estructura de directorios y archivos de una aplicación llamada ejemplo. Como puede observarse, tanto la aplicación ejemplo como la aplicación ejemplo2 están ubicadas dentro del directorio **webapps**:

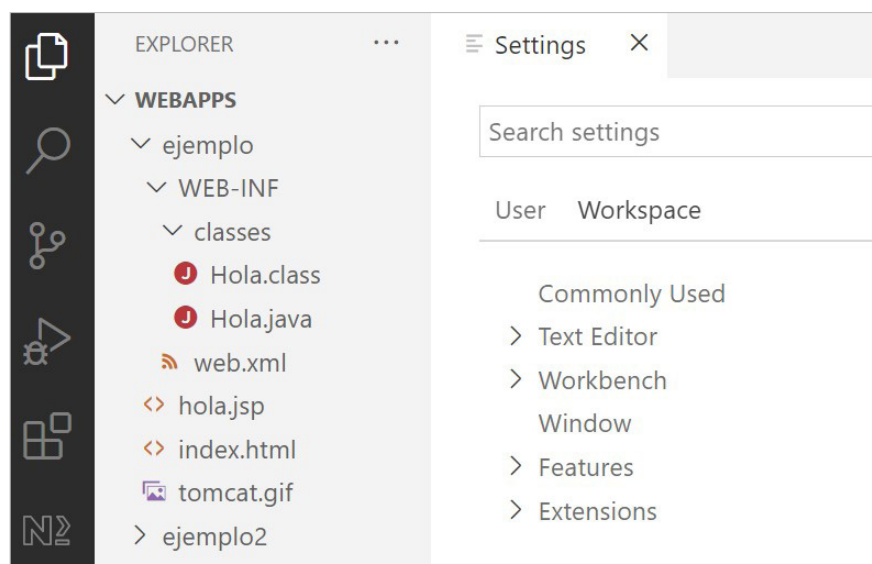


Figura 3. Directorio Webapps.

El archivo principal de la aplicación es **index.html**, el directorio **classes** contiene el archivo .java, con el código fuente, y el archivo compilado .class, con el **bytecode**. Para que la aplicación funcione, solamente es necesario tener en este directorio el archivo compilado .class, aunque, para que el desarrollo sea más cómodo, conviene generar los archivos .java directamente en ese directorio para que, al compilarlos, los archivos .class se generen directamente en la localización correcta.

A continuación se describen algunos de los métodos que se pueden utilizar para desplegar una aplicación en Tomcat:

- » Copiar el directorio completo de la aplicación descomprimida en el directorio `$CATALINA_BASE/webapps/`. Tomcat reconocerá la aplicación automáticamente.
- » Empaquetar el directorio de la aplicación en un archivo `.war` y copiar este archivo en el directorio `$CATALINA_BASE/webapps/`. Tomcat expandirá automáticamente el archivo y creará el directorio descomprimido.
- » Empaquetar el directorio de la aplicación en un archivo `.war` y utilizar el gestor de aplicaciones para cargarlo y desplegarlo. El efecto es el mismo que en el caso anterior. Tomcat descomprimirá el fichero `.war` y creará el directorio con todos los subdirectorios y archivos de la aplicación dentro del directorio `$CATALINA_BASE/webapps/`.

2.4. Gestor de aplicaciones de Tomcat

El gestor de aplicaciones de Tomcat es la herramienta que permite gestionar las aplicaciones que hay en el servidor. Para utilizarla, se debe acceder a la pantalla principal del servidor Tomcat, que es el que se muestra en la siguiente imagen:

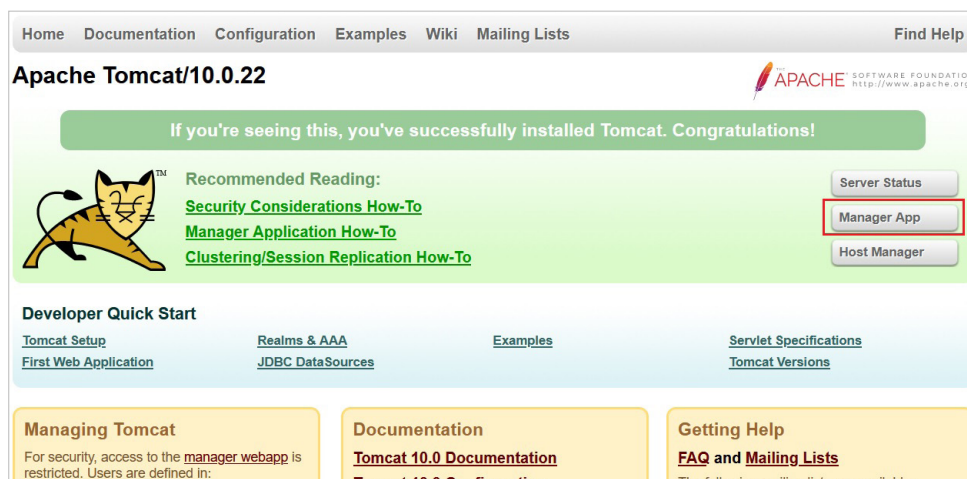


Figura 4. Pantalla de Tomcat.

Cuando estamos en la pantalla principal del servidor, hay que hacer clic en el botón *Manager App* para acceder al gestor de aplicaciones, que tiene este aspecto:




Gestor de Aplicaciones Web de Tomcat

Mensaje: OK - Parada aplicación en trayectoria de contexto [/ejemplo2]

Gestor

[Listar Aplicaciones](#)
[Ayuda HTML de Gestor](#)
[Ayuda de Gestor](#)
[Estado de Servidor](#)

Aplicaciones					
Ruta	Versión	Nombre a Mostrar	Ejecutándose	Sesiones	Comandos
/	Ninguno especificado	Welcome to Tomcat	true	0	Arrancar Parar Recargar Replegar Expirar sesiones sin trabajar ≥ 30 minutos
/docs	Ninguno especificado	Tomcat Documentation	true	0	Arrancar Parar Recargar Replegar Expirar sesiones sin trabajar ≥ 30 minutos
/ejemplo2	Ninguno especificado	Aplicación hola	false	0	Arrancar Parar Recargar Replegar
/examples	Ninguno especificado	Servlet and JSP Examples	true	0	Arrancar Parar Recargar Replegar Expirar sesiones sin trabajar ≥ 30 minutos
/host-manager	Ninguno especificado	Tomcat Host Manager Application	true	0	Arrancar Parar Recargar Replegar Expirar sesiones sin trabajar ≥ 30 minutos
/manager	Ninguno especificado	Tomcat Manager Application	true	1	Arrancar Parar Recargar Replegar Expirar sesiones sin trabajar ≥ 30 minutos

Figura 5. Pantalla de Tomcat.

En la imagen anterior puede verse el listado de las aplicaciones, una por cada directorio de los que hay dentro del directorio **webapps**.

Por cada aplicación aparece la ruta, la versión, el nombre, si está o no en ejecución, las sesiones y varios botones con los siguientes comandos:

» **Arrancar:** inicia la ejecución de la aplicación.

» **Parar:** detiene la ejecución de la aplicación.

» **Recargar:** esta opción se suele utilizar después de hacer cambios en algún archivo .java y crear su correspondiente archivo .class para liberar de caché la versión antigua.

» **Replegar:** esta última opción elimina el directorio de la aplicación del directorio **webapps**.

Inmediatamente después del listado de aplicaciones, se encuentra la sección *Desplegar*, desde la cual se puede cargar un archivo .war y, así, preparar fácilmente la aplicación para ejecutarla.

Desplegar

Desplegar directorio o archivo WAR localizado en servidor

Trayectoria de Contexto (opcional):
 Version (for parallel deployment):
 URL de archivo de Configuración XML:
 URL de WAR o Directorio:

Archivo WAR a desplegar

Seleccione archivo WAR a cargar No se ha seleccionado ningún archivo.

Figura 6. Pantalla de Tomcat.

2.5. JSP y servlets Java

Las páginas de servidor Java (JSP) y los servlets Java son programas escritos en lenguaje Java. La tarea principal de estos programas es añadir funcionalidades a los servicios que ofrece el servidor web. Generalmente, se utilizan para crear aplicaciones dinámicas, que son las que cargan la información en tiempo de ejecución, ya que son muy útiles a la hora de gestionar datos a través de operaciones de consulta, inserción, modificación o eliminación. Tanto las páginas de servidor Java (JSP) como los servlets Java deben ejecutarse sobre un servidor con capacidad de procesar aplicaciones Java.

Un servlet Java es una clase que extiende la clase Java `HttpServlet` y que tiene la capacidad de responder a solicitudes HTTP. Debe compilarse para que el servidor de aplicaciones pueda ejecutarlo. A continuación, se muestra un ejemplo de archivo .java con el código fuente antes de compilar:

```
import java.io.*;
import jakarta.servlet.*;
import jakarta.servlet.http.*;

public class Hola extends HttpServlet {

    public void doGet (HttpServletRequest
request,
HttpServletResponse
response)
throws IOException, ServletException
{
    response.setContentType("text/html");
    PrintWriter out = response.getWriter();
    out.println("<html>");
    out.println("<head>");
    out.println("<title>Hola</title>");
    out.println("</head>");
    out.println("<body>");
    out.println("<h1>Servlet de pruebas</h1>");
    out.println("</body>");
    out.println("</html>");
}
}
```

Los .jsp son archivos en los que se insertan porciones de código Java dentro del contenido HTML. Estos archivos .jsp se convierten en servlets antes de ser ejecutados por el servidor de aplicaciones. A continuación se muestra un ejemplo de archivo .jsp:

```
<%@ page contentType="text/html; charset=UTF-8" %>
<html>
<head>
    <meta charset="utf-8">
    <title>Ejemplo de página JSP</title>
</head>
<body bgcolor=white>
    <h1>Ejemplo de página JSP</h1> <br>
    

    <%= new String("Hola, esto es un ejemplo") %>
</body>
</html>
```

02 Apache Tomcat avanzado

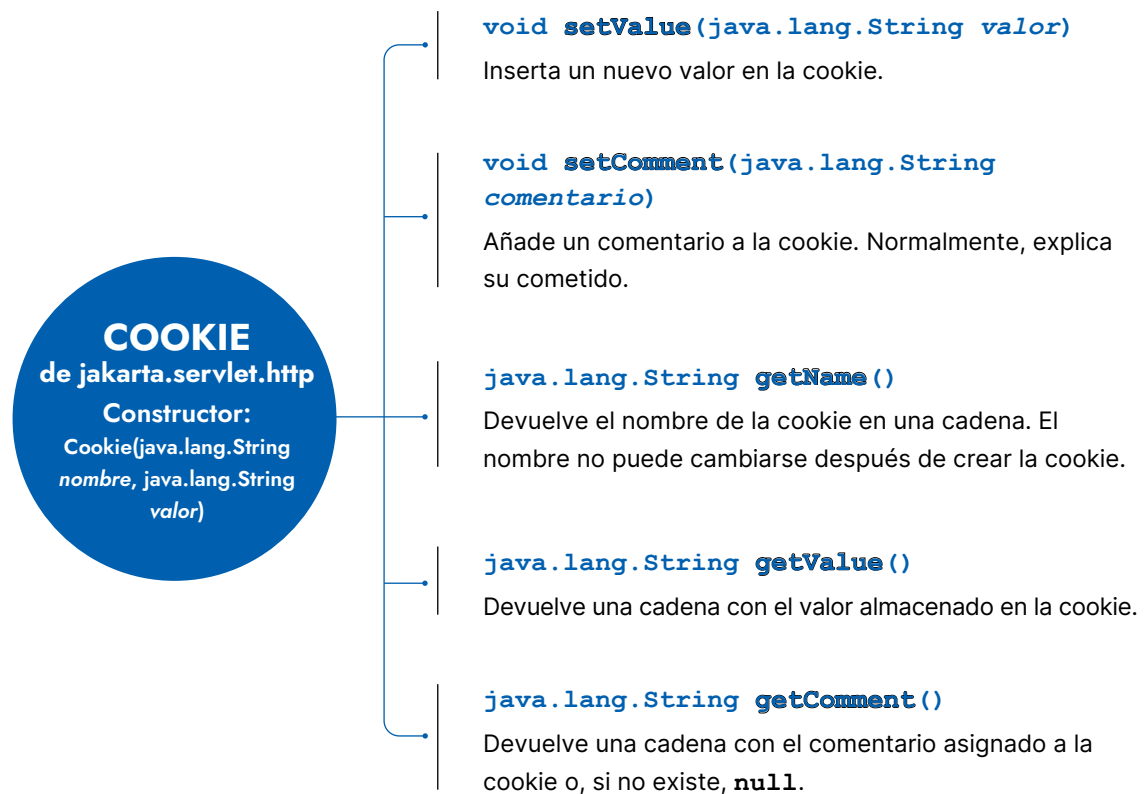
En este apartado se van a estudiar algunos aspectos más avanzados de la configuración de Apache Tomcat. Por un lado, se verá cómo Java permite gestionar las cookies y sesiones para que mejore la experiencia de los usuarios al utilizar las aplicaciones del servidor. Y por el otro lado, se trabajará la seguridad del servidor de aplicaciones, de manera que este se validará frente al cliente web y la información viajará cifrada.

3.1. Cookies

La actividad de un usuario sobre una aplicación o un sitio web es una **sesión**. Esta guarda las opciones que ha elegido el usuario, estados de la aplicación o cualquier otro dato que sea necesario para su funcionamiento. Esta información de usuario se puede almacenar a través de cookies o a través del uso de sesiones.

Las **cookies** son el mecanismo por el que un programa o servlet almacena pequeñas cantidades de información en archivos de texto en el equipo del usuario y, a su vez, el navegador web los envía al mismo servlet con cada nueva petición HTTP. Gracias a la información almacenada, el servidor es capaz de conocer ciertos aspectos de la actividad previa, lo que permite, entre otras cosas, la identificación y la personalización del contenido, incluso de los anuncios.

En Java, las cookies se gestionan a través de la clase `Cookie` del paquete `javax.servlet.http`. Una cookie tiene un nombre, un valor y una serie de atributos opcionales como un comentario o el tiempo máximo de vida. A continuación, se muestra el constructor de esta clase y algunos de sus métodos.



Ahora se presenta un ejemplo de servlet en el que se supone que una de las cookies determina el color del cuerpo de la página web que devolverá al cliente. Cabe destacar que el método `getCookies` de la interfaz `HttpServletRequest` devuelve un vector con todas las cookies.

```
// Se obtienen las cookies y se almacenan en un vector que se recorre
// buscando la cookie llamada "ColorFondo". Cuando se encuentra, se
// usa su valor para establecer el color del fondo.
import java.io.*;
import jakarta.servlet.*;
import jakarta.servlet.http.*;
public class EjemploCookie extends HttpServlet {
public void doGet(HttpServletRequest peti, HttpServletResponse resp)
    throws ServletException, IOException {
    resp.setContentType("text/html");
    PrintWriter out = resp.getWriter();

    // Tratamiento de las cookies
    Cookie[] cookies = peti.getCookies();
    String color = "";
    String nombre = "";
    [...]
```

```

    for (int i = 0; i < cookies.length; i++) {
        Cookie c = cookies[i];
        nombre = c.getName();
        if ( nombre == "ColorFondo" ) {
            color = c.getValue();
        }
        [...]
    }
    [...]
    out.println("<html><head><title>Ejemplo cookies</title></head>");
    out.println("<body bgcolor=\"\" + color + \"\">");
    [...]
}
}

```

3.2. Manejo de sesiones

Gracias a las sesiones, los programas o servlets pueden almacenar información, pero, a diferencia de las cookies, esta se almacena en el servidor. El uso de las sesiones permite identificar y almacenar información sobre los usuarios cuando realizan varias peticiones a un mismo servidor.

En una arquitectura Java, el contenedor de servlets puede **crear sesiones** a través del método `getSession` de la interfaz `HttpServletRequest`, mientras que, para **gestionirlas**, utiliza la interfaz `HttpSession`. Ambas son interfaces del paquete `javax.servlet.http`. Los servlets pueden consultar o establecer la información, así como vincular objetos a sesiones. A continuación, se muestran algunos métodos de `HttpSession` y un ejemplo de uso.

Interface `HttpSession` de `jakarta.servlet.http`

Métodos

```

void invalidate()
void setAttribute(java.lang.String nombre, java.lang.Object valor)
void removeAttribute(java.lang.String name)
java.util.Enumeration<java.lang.String> getAttributeNames()
java.lang.Object getAttribute(java.lang.String name)
java.lang.String getId()
long getCreationTime()
long getLastAccessedTime()

```

En este otro ejemplo de servlet es la información que almacena la sesión la que determina el color del cuerpo de la página web que devolverá al cliente.

```
// Se obtiene la sesión a la que pertenece la conexión, se obtienen
// sus atributos y se busca el llamado "ColorFondo", cuando se
// encuentra, se usa su valor para establecer el color del fondo.

import java.io.*;
import java.util.*;
import jakarta.servlet.*;
import jakarta.servlet.http.*;

public class EjemploSesion extends HttpServlet {

    public void doGet(HttpServletRequest peti, HttpServletResponse resp)
        throws ServletException, IOException {

        resp.setContentType("text/html");
        PrintWriter out = resp.getWriter();
        HttpSession session = peti.getSession(true);
        [...]

        // Contenido de la sesión
        Enumeration nombres = session.getAttributeNames();
        String color = "";
        String nombre = "";
        while (nombres.hasMoreElements()) {
            nombre = (String)nombres.nextElement();
            if ( nombre == "ColorFondo" ) {
                color = session.getAttribute(nombre).toString();
            }
            [...]
        }
        [...]
        out.println("<html><head><title>Ejemplo sesiones</title></head>");
        out.println("<body bgcolor=\"\" + color + \"\">");
        [...]
    }
}
```

3.3. Seguridad

La instalación por defecto de Tomcat ya incluye algunas medidas de seguridad, pero puede que no sean las apropiadas para todos los entornos de explotación o, ¿por qué no?, de desarrollo. Por esta razón debe realizarse un análisis para determinar la necesidad de medidas de seguridad adicionales en Tomcat o en el entorno donde se está ejecutando. Es por esto por lo que debe implementarse una **defensa en profundidad** en la que se establezcan varias capas o líneas de defensa. A continuación, se enumeran algunos elementos con ejemplos que podrían tenerse en cuenta:

- » **Cliente:** instalación de antivirus y formación sobre hábitos seguros.
- » **Perímetro de la red corporativa:** configuración de cortafuegos y uso de VPN (red privada virtual).
- » **Red interna y DMZ:** instalación de IDS/IPS, de cortafuegos y microsegmentación.
- » **Servidor:** actualización del sistema y configuración del cortafuegos.
- » **Tomcat:** parametrización y configuración de protocolos seguros.
- » **Datos:** copias de seguridad y cifrado.

A. Seguridad general de Tomcat

Para obtener un buen nivel de **seguridad en Tomcat**, se debería trabajar en varios aspectos de la aplicación, como, por ejemplo:

- » Ejecutar Tomcat con un usuario no administrador y siguiendo el principio de menor privilegio (por ejemplo, que no tenga acceso remoto al sistema).
- » Restringir los permisos sobre los archivos de la aplicación y los de trabajo, aunque la configuración posterior a la instalación por defecto suele ser suficiente.
- » Si no se van a usar, eliminar las aplicaciones y ejemplos proporcionados por Tomcat para evitar que se puedan aprovechar posibles vulnerabilidades.
- » Eliminar toda referencia a la versión de Tomcat utilizada para, así, dificultar el uso de programas maliciosos que explotan las vulnerabilidades. A estos programas se los conoce como *exploits*. Por esto, es recomendable eliminar la documentación de Tomcat y los ejemplos de los sistemas en producción.
- » Protección de las aplicaciones de administración usando nombres de usuario poco comunes y contraseñas seguras. También es importante no deshabilitar las protecciones contra ataques de fuerza bruta y, en caso de necesitar acceso remoto a estas aplicaciones, limitarlo a ciertas direcciones IP.

- » Activar el *Security manager* para que las aplicaciones se ejecuten en un *sandbox*, o entorno limitado, que rebaja significativamente la capacidad de las aplicaciones de efectuar cierto tipo de ataques contra el sistema. Hay que tener en cuenta que la activación del *Security manager* puede hacer que algunas aplicaciones no funcionen correctamente, por lo que se deben hacer pruebas antes de activarlo en un sistema en explotación.
- » Utilizar diferentes instancias de Tomcat, incluso hosts separados, si se ejecutan aplicaciones en las que no se confía. Así se reduce la posibilidad de ataques entre ellas.
- » Deshabilitar el puerto de apagado del servidor o protegerlo con una contraseña robusta.
- » Deshabilitar los conectores que no se utilicen y configurar correctamente los que se necesiten.
- » Habilitar SSL/TLS para la autenticación a través de redes no confiables, pero también con el resto de la sesión para evitar que la cookie de sesión sea interceptada.

B. Configuración de SSL/TLS

Como ya se estudió en la Unidad 2, el protocolo **HTTPS** aporta, a través del protocolo **TLS** (*Transport Layer Security*), autenticación del servidor frente al cliente, así como confidencialidad e integridad en las comunicaciones entre ellos. TLS trabaja entre los protocolos TCP y HTTP, y se basa en el **cifrado híbrido**, de manera que aprovecha tanto las ventajas del cifrado simétrico como las del asimétrico.

Como ya se ha explicado, es recomendable trabajar con SSL/TLS para proteger las comunicaciones, tanto las que comprenden la identificación y autenticación como el resto de las comunicaciones de la sesión.

Cuando Tomcat trabaja de manera independiente sin la necesidad de interactuar con un servidor web —es decir, en modo *stand-alone*—, se debe configurar SSL/TLS en él, mientras que, si trabaja detrás de un servidor web, lo normal es configurarlo en este último, ya que las comunicaciones con Tomcat suelen establecerse en un entorno seguro, bien porque estén en el mismo equipo, bien porque se realicen a través de redes de confianza. En el siguiente esquema pueden verse los dos modelos, aunque nos centraremos en el primero de ellos.

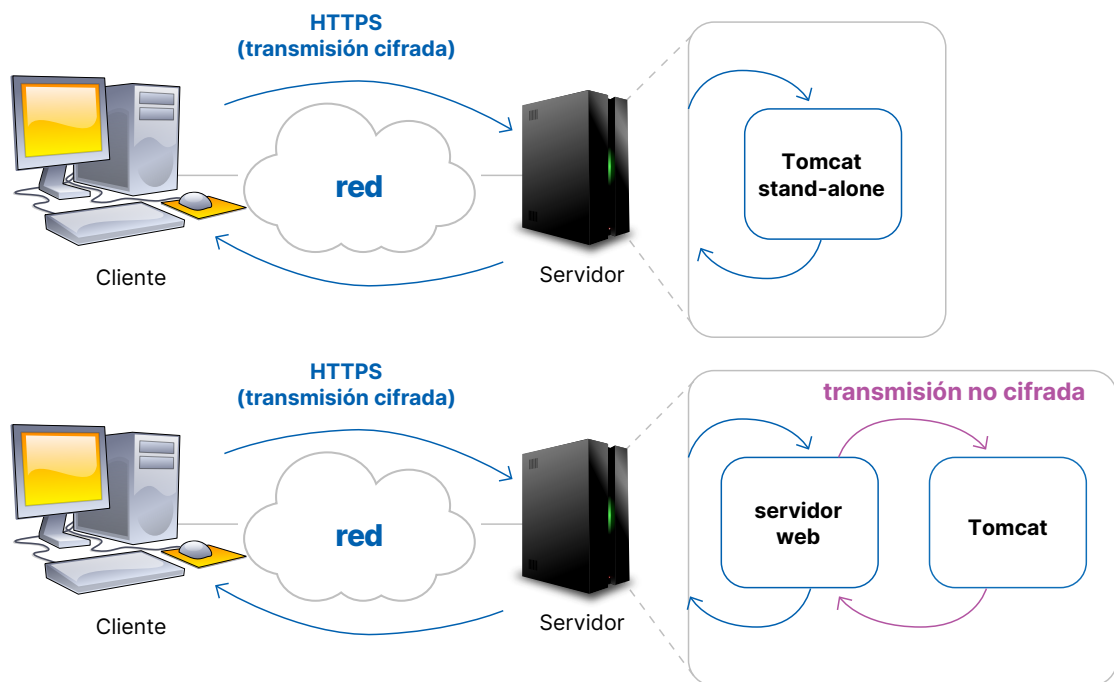
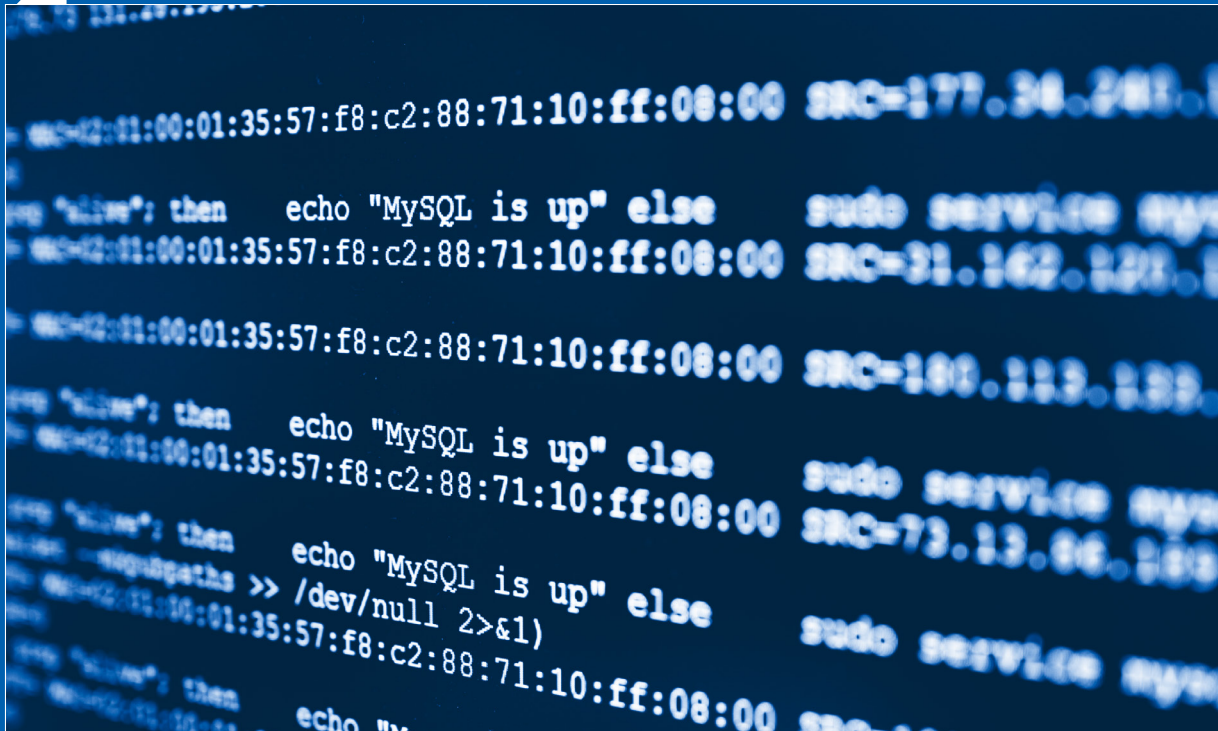


Figura 7. Transmisión de HTTPS

SSL/TLS se basa en pares de claves pública-privada y certificados, por lo que el servidor Tomcat necesita instalar al menos uno. Es importante recordar que, si se usa un certificado autofirmado —por ejemplo, con la herramienta de Tomcat llamada *keytool*—, los clientes podrían tener problemas para conectarse y, en consecuencia, para poder ejecutar las aplicaciones de un servidor seguro. Para evitar esta situación y conseguir que los clientes confíen en el certificado del servidor Tomcat, se debe recurrir a una autoridad certificadora (AC) que firme su clave pública. Si se trata de un servidor de uso interno, se puede recurrir a la instalación de una PKI (infraestructura de clave pública) propia y distribuir la clave pública de la AC entre todos los clientes para que reconozcan como legítimos todos los certificados firmados por ella. Por otra parte, si se trata de un servidor accesible por el público en general, debería estar firmado por una AC reconocida por la mayoría de los navegadores web.

2 CASO PRÁCTICO



GESTIÓN DE PREFERENCIAS DE USUARIO CON SERVLETS

En este caso práctico, vas a trabajar con un servlet que, gracias a las cookies, será capaz de gestionar alguna preferencia de usuario a la hora de visualizar la aplicación.

Estudia atentamente el código escrito en Java para:

- » Determinar qué bloque se ejecuta en la primera llamada, cuál en la segunda y, por último, el que se ejecutará en el resto de las llamadas. Puedes ejecutarlo para comprobar si estás en lo cierto.
- » Modificar el código fuente propuesto para que, además de establecer las preferencias del fondo de toda la página web, se pueda determinar el tamaño de la fuente del reloj. Ojo, solo de la fuente del reloj, no del resto de los elementos.

Recuerda que para ejecutar este servlet en Tomcat debes crear toda la estructura de directorios y archivos necesaria. Por otro lado, es aconsejable lo siguiente:

- » que te conectes con el usuario Tomcat;
- » que realices las pruebas con navegadores en los modos incógnito o navegación segura para evitar problemas de cookies almacenadas de otras sesiones;
- » si te surgen errores en tiempo de ejecución, el código fuente de las páginas HTML de salida y la información del directorio *logs* de Tomcat te pueden ayudar a solucionarlos.

SOLUCIÓN

Este es el orden de ejecución de los bloques en las diferentes llamadas:

- » **Primera llamada:** bloque B, porque no recibe el parámetro color, ni ningún otro, ni tampoco cookies. Este bloque muestra el formulario donde introducir el color de fondo deseado.
- » **Segunda llamada:** bloque A, puesto que desde la llamada anterior se le pasa el parámetro color. Este bloque crea y añade la cookie para que pueda ser usada en las siguientes llamadas.
- » **Resto de las llamadas:** bloque C, se ejecuta cuando no se pasa ningún color como parámetro y, además, existe una cookie.

A continuación, en el archivo descargable, hay un ejemplo de código con las modificaciones a *Fondo.java* para establecer el tamaño de la fuente del reloj. Date cuenta de que, en este caso, se establece gracias a las cabeceras de diferentes niveles. No obstante, este caso tiene muchas soluciones posibles.

UAX FP